

```

In [5]: import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
file_path = "C:/Users/Anarghya/OneDrive/Desktop/major project/energy_consumption_data.csv"
df = pd.read_csv(file_path)

# Encode categorical variable (Weather)
df["Weather"] = LabelEncoder().fit_transform(df["Weather"])

# Select features and target
X = df.drop(columns=["Total Energy Consumption (kWh)"])
y = df["Total Energy Consumption (kWh)"]

# Normalize data
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1))

# Reshape for LSTM (samples, time steps, features)
X_lstm = np.reshape(X_scaled, (X_scaled.shape[0], 1, X_scaled.shape[1]))

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_lstm, y_scaled, test_size=0.2)

# Define LSTM model
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(1, X_train.shape[2])),
    Dropout(0.2),
    LSTM(32, return_sequences=False),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1)
])

# Compile model
model.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae'])

# Train model with early stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
history = model.fit(X_train, y_train, epochs=100, batch_size=8, validation_data=(X_test, y_test))

# Predictions
y_pred_scaled = model.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred_scaled)
y_test_inv = scaler_y.inverse_transform(y_test)

# Display Prediction
print(y_pred)

# Evaluation Metrics
mse = mean_squared_error(y_test_inv, y_pred)

```

```
rmse = np.sqrt(mse)
r2 = r2_score(y_test_inv, y_pred)

print(f"\nMSE: {mse:.4f}\nRMSE: {rmse:.4f}\nR²: {r2:.4f}")

# Plot Predictions vs Actual
plt.figure(figsize=(10, 5))
plt.plot(y_test_inv, label="Actual", marker='o')
plt.plot(y_pred, label="Predicted", marker='x')
plt.xlabel("Sample")
plt.ylabel("Energy Consumption (kWh)")
plt.legend()
plt.title("Actual vs Predicted Energy Consumption")
plt.show()
```

C:\WINDOWS\system32\myenv\lib\site-packages\keras\src\layers\rnn\rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

```
Epoch 1/100
10/10  4s 66ms/step - loss: 0.2774 - mae: 0.4768 - val_loss: 0.3046 - val_mae: 0.4734
Epoch 2/100
10/10  0s 20ms/step - loss: 0.1915 - mae: 0.3771 - val_loss: 0.2095 - val_mae: 0.3818
Epoch 3/100
10/10  0s 13ms/step - loss: 0.1174 - mae: 0.2877 - val_loss: 0.1008 - val_mae: 0.2677
Epoch 4/100
10/10  0s 15ms/step - loss: 0.0658 - mae: 0.2063 - val_loss: 0.0640 - val_mae: 0.2213
Epoch 5/100
10/10  0s 18ms/step - loss: 0.0571 - mae: 0.1917 - val_loss: 0.0651 - val_mae: 0.2268
Epoch 6/100
10/10  0s 16ms/step - loss: 0.0494 - mae: 0.1776 - val_loss: 0.0711 - val_mae: 0.2364
Epoch 7/100
10/10  0s 13ms/step - loss: 0.0398 - mae: 0.1590 - val_loss: 0.0652 - val_mae: 0.2283
Epoch 8/100
10/10  0s 14ms/step - loss: 0.0374 - mae: 0.1557 - val_loss: 0.0613 - val_mae: 0.2221
Epoch 9/100
10/10  0s 13ms/step - loss: 0.0403 - mae: 0.1601 - val_loss: 0.0585 - val_mae: 0.2170
Epoch 10/100
10/10  0s 15ms/step - loss: 0.0342 - mae: 0.1474 - val_loss: 0.0565 - val_mae: 0.2136
Epoch 11/100
10/10  0s 20ms/step - loss: 0.0372 - mae: 0.1520 - val_loss: 0.0522 - val_mae: 0.2050
Epoch 12/100
10/10  0s 13ms/step - loss: 0.0380 - mae: 0.1651 - val_loss: 0.0498 - val_mae: 0.2004
Epoch 13/100
10/10  0s 18ms/step - loss: 0.0233 - mae: 0.1266 - val_loss: 0.0434 - val_mae: 0.1864
Epoch 14/100
10/10  0s 13ms/step - loss: 0.0274 - mae: 0.1368 - val_loss: 0.0369 - val_mae: 0.1693
Epoch 15/100
10/10  0s 11ms/step - loss: 0.0167 - mae: 0.1052 - val_loss: 0.0326 - val_mae: 0.1618
Epoch 16/100
10/10  0s 14ms/step - loss: 0.0189 - mae: 0.1032 - val_loss: 0.0249 - val_mae: 0.1399
Epoch 17/100
10/10  0s 16ms/step - loss: 0.0118 - mae: 0.0827 - val_loss: 0.0216 - val_mae: 0.1317
Epoch 18/100
10/10  0s 12ms/step - loss: 0.0104 - mae: 0.0828 - val_loss: 0.0162 - val_mae: 0.1134
Epoch 19/100
10/10  0s 12ms/step - loss: 0.0095 - mae: 0.0810 - val_loss: 0.0127 - val_mae: 0.0989
Epoch 20/100
10/10  0s 13ms/step - loss: 0.0064 - mae: 0.0569 - val_loss: 0.0112 - val_mae: 0.0943
Epoch 21/100
10/10  0s 14ms/step - loss: 0.0112 - mae: 0.0895 - val_loss: 0.0083 - val_mae: 0.0798
Epoch 22/100
```

10/10 ————— 0s 14ms/step - loss: 0.0088 - mae: 0.0798 - val_loss: 0.0085 - val_mae: 0.0805
Epoch 23/100

10/10 ————— 0s 12ms/step - loss: 0.0087 - mae: 0.0704 - val_loss: 0.0065 - val_mae: 0.0711
Epoch 24/100

10/10 ————— 0s 15ms/step - loss: 0.0058 - mae: 0.0596 - val_loss: 0.0068 - val_mae: 0.0723
Epoch 25/100

10/10 ————— 0s 13ms/step - loss: 0.0056 - mae: 0.0561 - val_loss: 0.0060 - val_mae: 0.0677
Epoch 26/100

10/10 ————— 0s 11ms/step - loss: 0.0046 - mae: 0.0559 - val_loss: 0.0072 - val_mae: 0.0730
Epoch 27/100

10/10 ————— 0s 11ms/step - loss: 0.0100 - mae: 0.0760 - val_loss: 0.0049 - val_mae: 0.0574
Epoch 28/100

10/10 ————— 0s 16ms/step - loss: 0.0067 - mae: 0.0602 - val_loss: 0.0057 - val_mae: 0.0643
Epoch 29/100

10/10 ————— 0s 11ms/step - loss: 0.0054 - mae: 0.0595 - val_loss: 0.0044 - val_mae: 0.0563
Epoch 30/100

10/10 ————— 0s 12ms/step - loss: 0.0059 - mae: 0.0638 - val_loss: 0.0074 - val_mae: 0.0720
Epoch 31/100

10/10 ————— 0s 12ms/step - loss: 0.0046 - mae: 0.0563 - val_loss: 0.0048 - val_mae: 0.0593
Epoch 32/100

10/10 ————— 0s 13ms/step - loss: 0.0057 - mae: 0.0591 - val_loss: 0.0043 - val_mae: 0.0557
Epoch 33/100

10/10 ————— 0s 12ms/step - loss: 0.0053 - mae: 0.0616 - val_loss: 0.0052 - val_mae: 0.0604
Epoch 34/100

10/10 ————— 0s 16ms/step - loss: 0.0047 - mae: 0.0585 - val_loss: 0.0033 - val_mae: 0.0477
Epoch 35/100

10/10 ————— 0s 16ms/step - loss: 0.0087 - mae: 0.0703 - val_loss: 0.0037 - val_mae: 0.0498
Epoch 36/100

10/10 ————— 0s 13ms/step - loss: 0.0038 - mae: 0.0525 - val_loss: 0.0028 - val_mae: 0.0427
Epoch 37/100

10/10 ————— 0s 13ms/step - loss: 0.0062 - mae: 0.0610 - val_loss: 0.0038 - val_mae: 0.0513
Epoch 38/100

10/10 ————— 0s 11ms/step - loss: 0.0035 - mae: 0.0427 - val_loss: 0.0033 - val_mae: 0.0479
Epoch 39/100

10/10 ————— 0s 12ms/step - loss: 0.0060 - mae: 0.0569 - val_loss: 0.0044 - val_mae: 0.0557
Epoch 40/100

10/10 ————— 0s 11ms/step - loss: 0.0051 - mae: 0.0559 - val_loss: 0.0048 - val_mae: 0.0578
Epoch 41/100

10/10 ————— 0s 13ms/step - loss: 0.0041 - mae: 0.0518 - val_loss: 0.0030 - val_mae: 0.0446
Epoch 42/100

10/10 ————— 0s 12ms/step - loss: 0.0042 - mae: 0.0538 - val_loss: 0.0046 - val_mae: 0.0574
Epoch 43/100

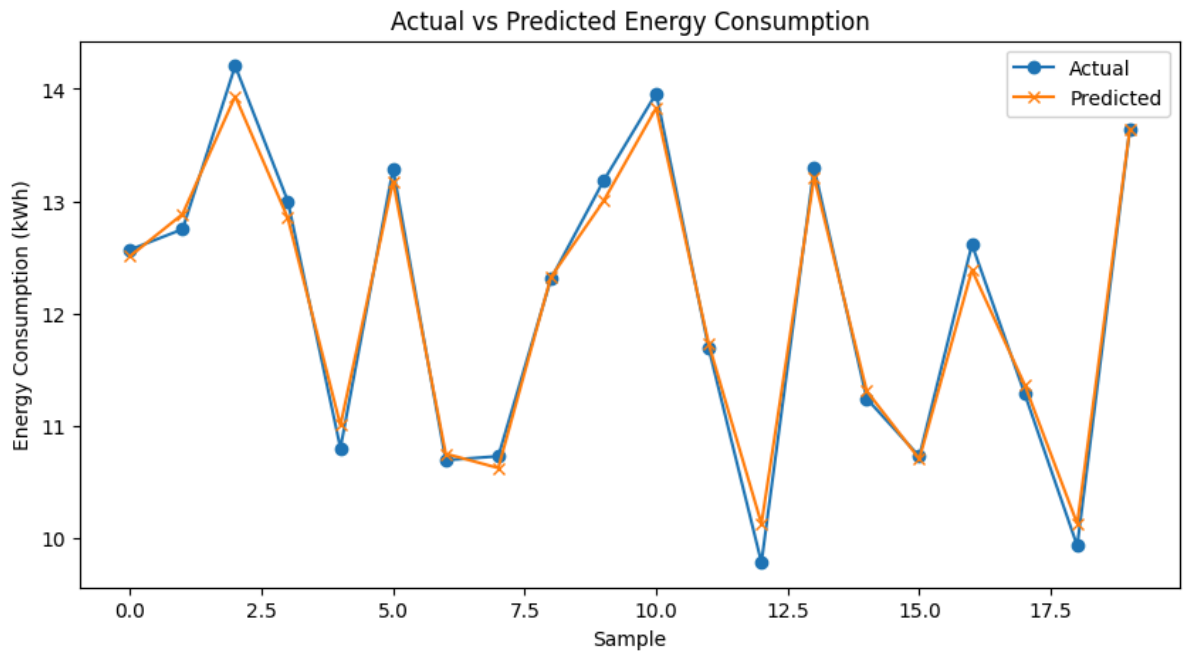
10/10 ————— 0s 11ms/step - loss: 0.0033 - mae: 0.0438 - val_loss:

```
0.0027 - val_mae: 0.0413
Epoch 44/100
10/10 ----- 0s 12ms/step - loss: 0.0047 - mae: 0.0555 - val_loss:
0.0028 - val_mae: 0.0422
Epoch 45/100
10/10 ----- 0s 11ms/step - loss: 0.0048 - mae: 0.0555 - val_loss:
0.0041 - val_mae: 0.0541
Epoch 46/100
10/10 ----- 0s 11ms/step - loss: 0.0057 - mae: 0.0552 - val_loss:
0.0027 - val_mae: 0.0423
Epoch 47/100
10/10 ----- 0s 11ms/step - loss: 0.0080 - mae: 0.0673 - val_loss:
0.0047 - val_mae: 0.0593
Epoch 48/100
10/10 ----- 0s 12ms/step - loss: 0.0046 - mae: 0.0543 - val_loss:
0.0030 - val_mae: 0.0452
Epoch 49/100
10/10 ----- 0s 11ms/step - loss: 0.0033 - mae: 0.0438 - val_loss:
0.0032 - val_mae: 0.0471
Epoch 50/100
10/10 ----- 0s 11ms/step - loss: 0.0036 - mae: 0.0491 - val_loss:
0.0023 - val_mae: 0.0389
Epoch 51/100
10/10 ----- 0s 11ms/step - loss: 0.0048 - mae: 0.0582 - val_loss:
0.0021 - val_mae: 0.0356
Epoch 52/100
10/10 ----- 0s 11ms/step - loss: 0.0045 - mae: 0.0542 - val_loss:
0.0040 - val_mae: 0.0540
Epoch 53/100
10/10 ----- 0s 12ms/step - loss: 0.0048 - mae: 0.0546 - val_loss:
0.0022 - val_mae: 0.0357
Epoch 54/100
10/10 ----- 0s 12ms/step - loss: 0.0045 - mae: 0.0554 - val_loss:
0.0029 - val_mae: 0.0461
Epoch 55/100
10/10 ----- 0s 12ms/step - loss: 0.0036 - mae: 0.0481 - val_loss:
0.0021 - val_mae: 0.0376
Epoch 56/100
10/10 ----- 0s 13ms/step - loss: 0.0035 - mae: 0.0455 - val_loss:
0.0028 - val_mae: 0.0434
Epoch 57/100
10/10 ----- 0s 11ms/step - loss: 0.0051 - mae: 0.0516 - val_loss:
0.0029 - val_mae: 0.0452
Epoch 58/100
10/10 ----- 0s 11ms/step - loss: 0.0034 - mae: 0.0446 - val_loss:
0.0023 - val_mae: 0.0408
Epoch 59/100
10/10 ----- 0s 11ms/step - loss: 0.0039 - mae: 0.0505 - val_loss:
0.0022 - val_mae: 0.0395
Epoch 60/100
10/10 ----- 0s 11ms/step - loss: 0.0033 - mae: 0.0447 - val_loss:
0.0019 - val_mae: 0.0362
Epoch 61/100
10/10 ----- 0s 13ms/step - loss: 0.0047 - mae: 0.0547 - val_loss:
0.0025 - val_mae: 0.0418
Epoch 62/100
10/10 ----- 0s 13ms/step - loss: 0.0033 - mae: 0.0456 - val_loss:
0.0018 - val_mae: 0.0343
Epoch 63/100
10/10 ----- 0s 13ms/step - loss: 0.0039 - mae: 0.0485 - val_loss:
0.0027 - val_mae: 0.0437
Epoch 64/100
10/10 ----- 0s 13ms/step - loss: 0.0035 - mae: 0.0447 - val_loss:
0.0019 - val_mae: 0.0351
```

Epoch 65/100
10/10 ————— 0s 14ms/step - loss: 0.0030 - mae: 0.0421 - val_loss: 0.0028 - val_mae: 0.0449
Epoch 66/100
10/10 ————— 0s 14ms/step - loss: 0.0049 - mae: 0.0544 - val_loss: 0.0020 - val_mae: 0.0345
Epoch 67/100
10/10 ————— 0s 14ms/step - loss: 0.0044 - mae: 0.0517 - val_loss: 0.0029 - val_mae: 0.0460
Epoch 68/100
10/10 ————— 0s 11ms/step - loss: 0.0037 - mae: 0.0468 - val_loss: 0.0025 - val_mae: 0.0407
Epoch 69/100
10/10 ————— 0s 12ms/step - loss: 0.0044 - mae: 0.0491 - val_loss: 0.0030 - val_mae: 0.0445
Epoch 70/100
10/10 ————— 0s 11ms/step - loss: 0.0038 - mae: 0.0492 - val_loss: 0.0023 - val_mae: 0.0416
Epoch 71/100
10/10 ————— 0s 11ms/step - loss: 0.0042 - mae: 0.0514 - val_loss: 0.0025 - val_mae: 0.0421
Epoch 72/100
10/10 ————— 0s 11ms/step - loss: 0.0023 - mae: 0.0388 - val_loss: 0.0014 - val_mae: 0.0310
Epoch 73/100
10/10 ————— 0s 12ms/step - loss: 0.0028 - mae: 0.0372 - val_loss: 0.0026 - val_mae: 0.0439
Epoch 74/100
10/10 ————— 0s 11ms/step - loss: 0.0039 - mae: 0.0489 - val_loss: 0.0018 - val_mae: 0.0364
Epoch 75/100
10/10 ————— 0s 12ms/step - loss: 0.0037 - mae: 0.0444 - val_loss: 0.0030 - val_mae: 0.0476
Epoch 76/100
10/10 ————— 0s 27ms/step - loss: 0.0034 - mae: 0.0426 - val_loss: 0.0014 - val_mae: 0.0282
Epoch 77/100
10/10 ————— 0s 15ms/step - loss: 0.0029 - mae: 0.0403 - val_loss: 0.0029 - val_mae: 0.0464
Epoch 78/100
10/10 ————— 0s 12ms/step - loss: 0.0037 - mae: 0.0488 - val_loss: 0.0017 - val_mae: 0.0333
Epoch 79/100
10/10 ————— 0s 26ms/step - loss: 0.0027 - mae: 0.0414 - val_loss: 0.0017 - val_mae: 0.0349
Epoch 80/100
10/10 ————— 0s 14ms/step - loss: 0.0040 - mae: 0.0529 - val_loss: 0.0024 - val_mae: 0.0404
Epoch 81/100
10/10 ————— 0s 13ms/step - loss: 0.0029 - mae: 0.0410 - val_loss: 0.0011 - val_mae: 0.0267
Epoch 82/100
10/10 ————— 0s 27ms/step - loss: 0.0038 - mae: 0.0467 - val_loss: 0.0016 - val_mae: 0.0326
Epoch 83/100
10/10 ————— 0s 14ms/step - loss: 0.0041 - mae: 0.0511 - val_loss: 0.0011 - val_mae: 0.0267
Epoch 84/100
10/10 ————— 0s 12ms/step - loss: 0.0034 - mae: 0.0473 - val_loss: 0.0017 - val_mae: 0.0336
Epoch 85/100
10/10 ————— 0s 12ms/step - loss: 0.0023 - mae: 0.0367 - val_loss: 0.0022 - val_mae: 0.0381
Epoch 86/100

```
10/10 ----- 0s 13ms/step - loss: 0.0037 - mae: 0.0438 - val_loss:
0.0014 - val_mae: 0.0298
Epoch 87/100
10/10 ----- 0s 12ms/step - loss: 0.0038 - mae: 0.0481 - val_loss:
0.0024 - val_mae: 0.0426
Epoch 88/100
10/10 ----- 0s 14ms/step - loss: 0.0035 - mae: 0.0439 - val_loss:
0.0012 - val_mae: 0.0299
Epoch 89/100
10/10 ----- 0s 15ms/step - loss: 0.0029 - mae: 0.0445 - val_loss:
0.0015 - val_mae: 0.0332
Epoch 90/100
10/10 ----- 0s 14ms/step - loss: 0.0019 - mae: 0.0351 - val_loss:
0.0016 - val_mae: 0.0343
Epoch 91/100
10/10 ----- 0s 12ms/step - loss: 0.0029 - mae: 0.0432 - val_loss:
0.0012 - val_mae: 0.0255
Epoch 92/100
10/10 ----- 0s 11ms/step - loss: 0.0043 - mae: 0.0513 - val_loss:
0.0020 - val_mae: 0.0375
Epoch 93/100
10/10 ----- 0s 11ms/step - loss: 0.0026 - mae: 0.0368 - val_loss:
0.0030 - val_mae: 0.0476
1/1 ----- 1s 605ms/step
[[12.510492 ]
 [12.885241 ]
 [13.93384  ]
 [12.850216 ]
 [11.006291 ]
 [13.176416 ]
 [10.752556 ]
 [10.625749 ]
 [12.32503  ]
 [13.002061 ]
 [13.830391 ]
 [11.726655]
 [10.129124 ]
 [13.215214 ]
 [11.3106165]
 [10.707023 ]
 [12.3878355]
 [11.367642 ]
 [10.129772 ]
 [13.643258 ]]

MSE: 0.0232
RMSE: 0.1523
R2: 0.9868
```



```
In [6]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report

# Define energy consumption categories
def categorize_energy(value, low_thresh, high_thresh):
    if value < low_thresh:
        return "Low"
    elif value < high_thresh:
        return "Medium"
    else:
        return "High"

# Define thresholds based on percentiles
low_threshold = np.percentile(y_test_inv, 33)
high_threshold = np.percentile(y_test_inv, 66)

# Convert actual and predicted values to categories
y_test_class = np.array([categorize_energy(val, low_threshold, high_threshold) for val in y_test_inv])
y_pred_class = np.array([categorize_energy(val, low_threshold, high_threshold) for val in y_pred_inv])

# Create confusion matrix
conf_matrix = confusion_matrix(y_test_class, y_pred_class, labels=["Low", "Medium", "High"])

# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=["Low", "Medium", "High"], yticklabels=["Low", "Medium", "High"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix for Energy Consumption Categories")
plt.show()

# Print classification report
print(classification_report(y_test_class, y_pred_class))
```



```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[6], line 3
      1 import numpy as np
      2 import pandas as pd
----> 3 import seaborn as sns
      4 import matplotlib.pyplot as plt
      5 from sklearn.metrics import confusion_matrix, classification_report

ModuleNotFoundError: No module named 'seaborn'
```

```
In [3]: import pandas as pd
import numpy as np
from sklearn.feature_selection import mutual_info_regression
import matplotlib.pyplot as plt

# Load dataset
file_path = "C:/Users/Anarghya/OneDrive/Desktop/major project/energy_consumption_data.csv"
df = pd.read_csv(file_path)

# Encode categorical variable (if any)
df["Weather"] = df["Weather"].astype("category").cat.codes # Convert categorical to numerical

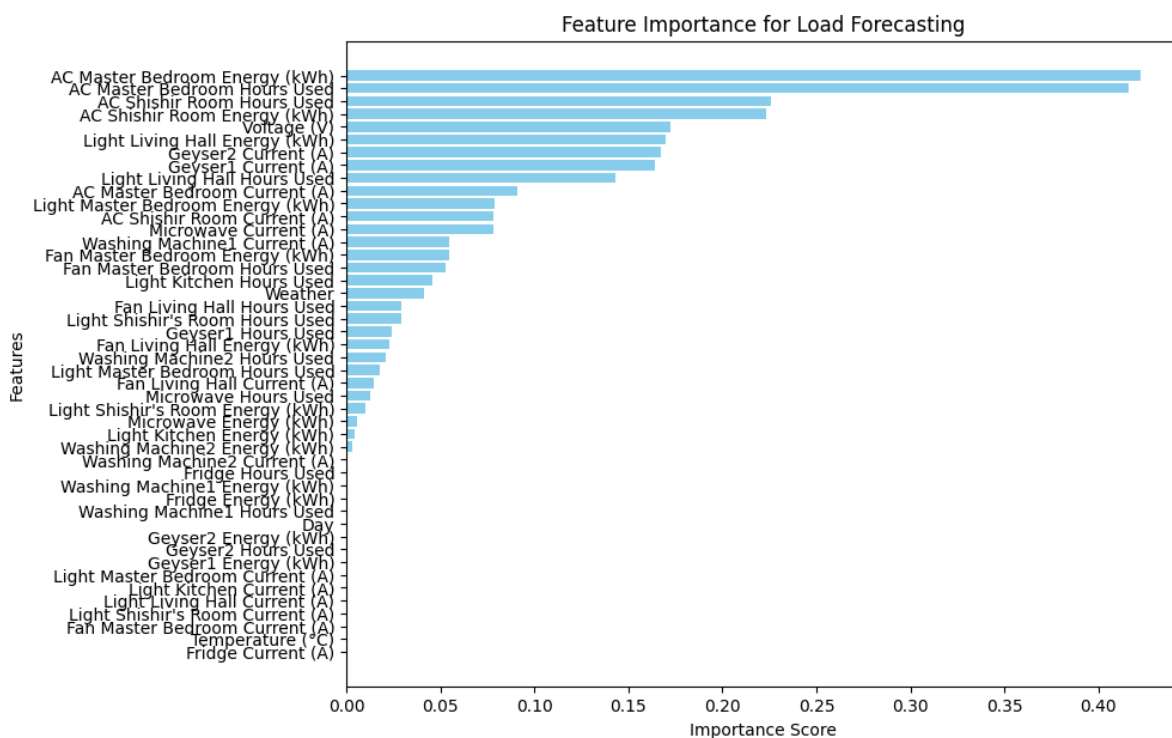
# Select features and target
X = df.drop(columns=["Total Energy Consumption (kWh)"]) # Features
y = df["Total Energy Consumption (kWh)"] # Target variable

# Compute feature importance using Mutual Information
feature_importance = mutual_info_regression(X, y)

# Create a DataFrame to store and sort feature importance
feature_importance_df = pd.DataFrame({'Feature': X.columns, 'Importance': feature_importance})
feature_importance_df = feature_importance_df.sort_values(by="Importance", ascending=False)

# Plot feature importance
plt.figure(figsize=(9, 7))
plt.barh(feature_importance_df["Feature"], feature_importance_df["Importance"], color="red")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.title("Feature Importance for Load Forecasting")
plt.gca().invert_yaxis()
plt.show()

# Display ranked features
print(feature_importance_df)
```



	Feature	Importance
8	AC Master Bedroom Energy (kWh)	0.422772
7	AC Master Bedroom Hours Used	0.416392
4	AC Shishir Room Hours Used	0.225544
5	AC Shishir Room Energy (kWh)	0.223232
3	Voltage (V)	0.172629
20	Light Living Hall Energy (kWh)	0.169858
36	Geyser2 Current (A)	0.167412
33	Geyser1 Current (A)	0.164274
19	Light Living Hall Hours Used	0.142794
9	AC Master Bedroom Current (A)	0.090783
26	Light Master Bedroom Energy (kWh)	0.078976
6	AC Shishir Room Current (A)	0.078365
30	Microwave Current (A)	0.077879
39	Washing Machine1 Current (A)	0.054824
14	Fan Master Bedroom Energy (kWh)	0.054606
13	Fan Master Bedroom Hours Used	0.052766
22	Light Kitchen Hours Used	0.045841
1	Weather	0.041198
10	Fan Living Hall Hours Used	0.029417
16	Light Shishir's Room Hours Used	0.029279
31	Geyser1 Hours Used	0.024300
11	Fan Living Hall Energy (kWh)	0.022620
40	Washing Machine2 Hours Used	0.020608
25	Light Master Bedroom Hours Used	0.017769
12	Fan Living Hall Current (A)	0.014755
28	Microwave Hours Used	0.012941
17	Light Shishir's Room Energy (kWh)	0.010044
29	Microwave Energy (kWh)	0.005438
23	Light Kitchen Energy (kWh)	0.004462
41	Washing Machine2 Energy (kWh)	0.003394
42	Washing Machine2 Current (A)	0.000000
43	Fridge Hours Used	0.000000
38	Washing Machine1 Energy (kWh)	0.000000
44	Fridge Energy (kWh)	0.000000
37	Washing Machine1 Hours Used	0.000000
0	Day	0.000000
35	Geyser2 Energy (kWh)	0.000000
34	Geyser2 Hours Used	0.000000
32	Geyser1 Energy (kWh)	0.000000
27	Light Master Bedroom Current (A)	0.000000
24	Light Kitchen Current (A)	0.000000
21	Light Living Hall Current (A)	0.000000
18	Light Shishir's Room Current (A)	0.000000
15	Fan Master Bedroom Current (A)	0.000000
2	Temperature (°C)	0.000000
45	Fridge Current (A)	0.000000

```
In [4]: import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import mean_absolute_error

# Load dataset
file_path = "C:/Users/Anarghya/OneDrive/Desktop/major project/energy_consumption_da
```

```

df = pd.read_csv(file_path)

# Encode categorical variable (Weather)
df["Weather"] = LabelEncoder().fit_transform(df["Weather"])

# Select features and target
X = df.drop(columns=["Total Energy Consumption (kWh)"])
y = df["Total Energy Consumption (kWh)"]

# Normalize data
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1))

# Reshape for LSTM (samples, time steps, features)
X_lstm = np.reshape(X_scaled, (X_scaled.shape[0], 1, X_scaled.shape[1]))

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_lstm, y_scaled, test_size=0.2)

# Define LSTM model
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(1, X_train.shape[2])),
    Dropout(0.2),
    LSTM(32, return_sequences=False),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1)
])

# Compile model
model.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae'])

# Train model with early stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
history = model.fit(X_train, y_train, epochs=100, batch_size=8, validation_data=(X_test, y_test))

# Predictions
y_pred_scaled = model.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred_scaled)
y_test_inv = scaler_y.inverse_transform(y_test)

# Display Prediction
print(y_pred)

# Evaluation Metrics
mse = mean_squared_error(y_test_inv, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_inv, y_pred)
mae = mean_absolute_error(y_test_inv, y_pred)

print(f"\nMSE: {mse:.4f}\nRMSE: {rmse:.4f}\nR²: {r2:.4f}\nMAE: {mae:.4f}")

plt.figure(figsize=(8, 4))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Training vs Validation Loss')
plt.show()

```

Epoch 1/100

```
C:\WINDOWS\system32\myenv\lib\site-packages\keras\src\layers\rnn\rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.  
    super().__init__(**kwargs)
```

10/10  3s 61ms/step - loss: 0.2171 - mae: 0.4176 - val_loss: 0.2304 - val_mae: 0.4054
Epoch 2/100

10/10  0s 14ms/step - loss: 0.1502 - mae: 0.3319 - val_loss: 0.1032 - val_mae: 0.2687
Epoch 3/100

10/10  0s 12ms/step - loss: 0.0615 - mae: 0.1947 - val_loss: 0.0588 - val_mae: 0.2103
Epoch 4/100

10/10  0s 17ms/step - loss: 0.0488 - mae: 0.1812 - val_loss: 0.0586 - val_mae: 0.2114
Epoch 5/100

10/10  0s 11ms/step - loss: 0.0515 - mae: 0.1781 - val_loss: 0.0696 - val_mae: 0.2336
Epoch 6/100

10/10  0s 11ms/step - loss: 0.0408 - mae: 0.1671 - val_loss: 0.0623 - val_mae: 0.2226
Epoch 7/100

10/10  0s 13ms/step - loss: 0.0476 - mae: 0.1688 - val_loss: 0.0550 - val_mae: 0.2078
Epoch 8/100

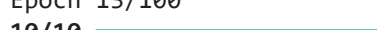
10/10  0s 11ms/step - loss: 0.0429 - mae: 0.1673 - val_loss: 0.0542 - val_mae: 0.2071
Epoch 9/100

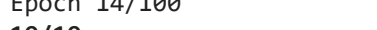
10/10  0s 11ms/step - loss: 0.0371 - mae: 0.1508 - val_loss: 0.0505 - val_mae: 0.2006
Epoch 10/100

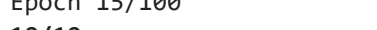
10/10  0s 12ms/step - loss: 0.0374 - mae: 0.1552 - val_loss: 0.0449 - val_mae: 0.1879
Epoch 11/100

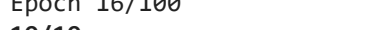
10/10  0s 11ms/step - loss: 0.0292 - mae: 0.1370 - val_loss: 0.0438 - val_mae: 0.1863
Epoch 12/100

10/10  0s 11ms/step - loss: 0.0316 - mae: 0.1431 - val_loss: 0.0358 - val_mae: 0.1675
Epoch 13/100

10/10  0s 10ms/step - loss: 0.0247 - mae: 0.1277 - val_loss: 0.0315 - val_mae: 0.1585
Epoch 14/100

10/10  0s 12ms/step - loss: 0.0140 - mae: 0.0923 - val_loss: 0.0240 - val_mae: 0.1379
Epoch 15/100

10/10  0s 15ms/step - loss: 0.0138 - mae: 0.1007 - val_loss: 0.0187 - val_mae: 0.1213
Epoch 16/100

10/10  0s 14ms/step - loss: 0.0135 - mae: 0.0923 - val_loss: 0.0138 - val_mae: 0.1010
Epoch 17/100

10/10  0s 14ms/step - loss: 0.0081 - mae: 0.0722 - val_loss: 0.0102 - val_mae: 0.0851
Epoch 18/100

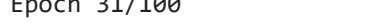
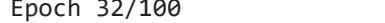
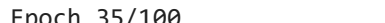
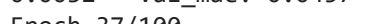
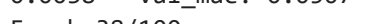
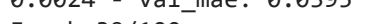
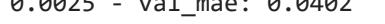
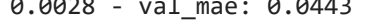
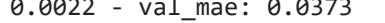
10/10  0s 14ms/step - loss: 0.0122 - mae: 0.0918 - val_loss: 0.0094 - val_mae: 0.0829
Epoch 19/100

10/10  0s 13ms/step - loss: 0.0095 - mae: 0.0845 - val_loss: 0.0072 - val_mae: 0.0709
Epoch 20/100

10/10  0s 10ms/step - loss: 0.0071 - mae: 0.0717 - val_loss: 0.0060 - val_mae: 0.0623
Epoch 21/100

10/10  0s 11ms/step - loss: 0.0059 - mae: 0.0600 - val_loss: 0.0063 - val_mae: 0.0669
Epoch 22/100

10/10  0s 12ms/step - loss: 0.0068 - mae: 0.0649 - val_loss:

```
0.0052 - val_mae: 0.0596
Epoch 23/100
10/10  0s 11ms/step - loss: 0.0063 - mae: 0.0663 - val_loss:
0.0042 - val_mae: 0.0490
Epoch 24/100
10/10  0s 11ms/step - loss: 0.0077 - mae: 0.0694 - val_loss:
0.0049 - val_mae: 0.0597
Epoch 25/100
10/10  0s 14ms/step - loss: 0.0069 - mae: 0.0660 - val_loss:
0.0038 - val_mae: 0.0487
Epoch 26/100
10/10  0s 11ms/step - loss: 0.0054 - mae: 0.0531 - val_loss:
0.0037 - val_mae: 0.0496
Epoch 27/100
10/10  0s 12ms/step - loss: 0.0042 - mae: 0.0516 - val_loss:
0.0033 - val_mae: 0.0444
Epoch 28/100
10/10  0s 12ms/step - loss: 0.0035 - mae: 0.0473 - val_loss:
0.0038 - val_mae: 0.0509
Epoch 29/100
10/10  0s 12ms/step - loss: 0.0048 - mae: 0.0593 - val_loss:
0.0031 - val_mae: 0.0449
Epoch 30/100
10/10  0s 11ms/step - loss: 0.0048 - mae: 0.0563 - val_loss:
0.0031 - val_mae: 0.0449
Epoch 31/100
10/10  0s 12ms/step - loss: 0.0066 - mae: 0.0654 - val_loss:
0.0031 - val_mae: 0.0452
Epoch 32/100
10/10  0s 10ms/step - loss: 0.0058 - mae: 0.0618 - val_loss:
0.0066 - val_mae: 0.0686
Epoch 33/100
10/10  0s 11ms/step - loss: 0.0038 - mae: 0.0475 - val_loss:
0.0038 - val_mae: 0.0518
Epoch 34/100
10/10  0s 12ms/step - loss: 0.0065 - mae: 0.0617 - val_loss:
0.0052 - val_mae: 0.0597
Epoch 35/100
10/10  0s 12ms/step - loss: 0.0065 - mae: 0.0613 - val_loss:
0.0038 - val_mae: 0.0524
Epoch 36/100
10/10  0s 12ms/step - loss: 0.0051 - mae: 0.0567 - val_loss:
0.0032 - val_mae: 0.0457
Epoch 37/100
10/10  0s 12ms/step - loss: 0.0059 - mae: 0.0613 - val_loss:
0.0038 - val_mae: 0.0507
Epoch 38/100
10/10  0s 12ms/step - loss: 0.0061 - mae: 0.0604 - val_loss:
0.0024 - val_mae: 0.0395
Epoch 39/100
10/10  0s 12ms/step - loss: 0.0040 - mae: 0.0473 - val_loss:
0.0025 - val_mae: 0.0402
Epoch 40/100
10/10  0s 11ms/step - loss: 0.0056 - mae: 0.0605 - val_loss:
0.0028 - val_mae: 0.0443
Epoch 41/100
10/10  0s 12ms/step - loss: 0.0035 - mae: 0.0465 - val_loss:
0.0022 - val_mae: 0.0373
Epoch 42/100
10/10  0s 10ms/step - loss: 0.0038 - mae: 0.0469 - val_loss:
0.0027 - val_mae: 0.0424
Epoch 43/100
10/10  0s 11ms/step - loss: 0.0039 - mae: 0.0526 - val_loss:
0.0023 - val_mae: 0.0406
```

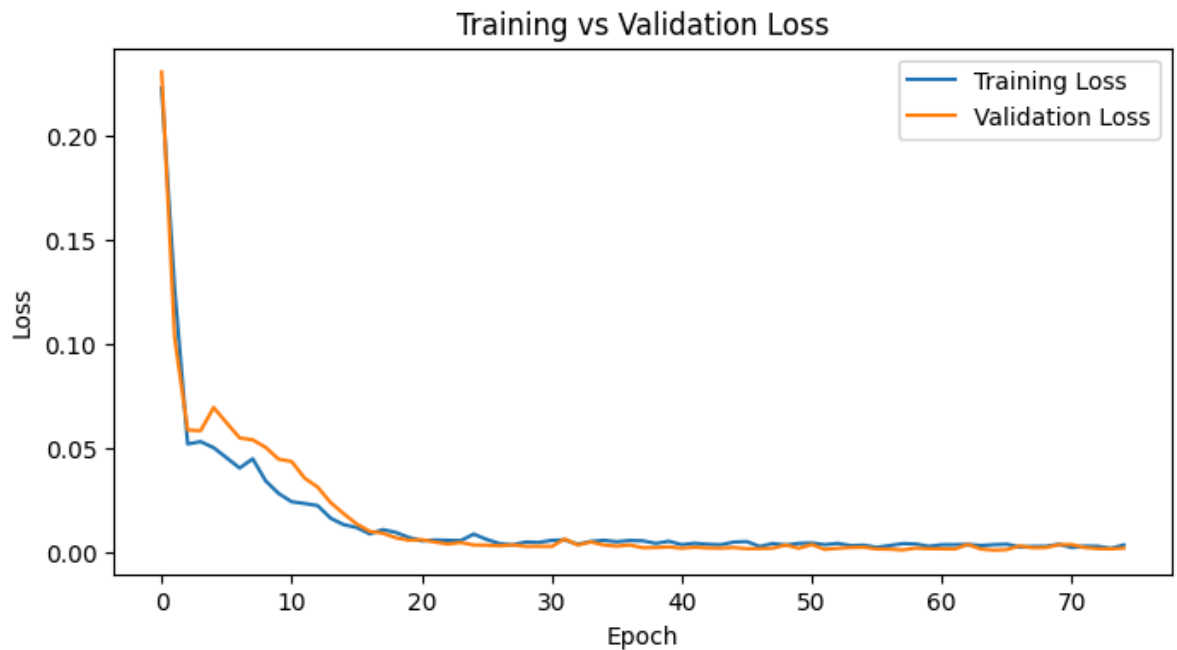
Epoch 44/100
10/10 ————— 0s 12ms/step - loss: 0.0039 - mae: 0.0509 - val_loss: 0.0022 - val_mae: 0.0378
Epoch 45/100
10/10 ————— 0s 10ms/step - loss: 0.0049 - mae: 0.0531 - val_loss: 0.0025 - val_mae: 0.0404
Epoch 46/100
10/10 ————— 0s 11ms/step - loss: 0.0058 - mae: 0.0611 - val_loss: 0.0020 - val_mae: 0.0363
Epoch 47/100
10/10 ————— 0s 12ms/step - loss: 0.0033 - mae: 0.0446 - val_loss: 0.0020 - val_mae: 0.0371
Epoch 48/100
10/10 ————— 0s 11ms/step - loss: 0.0040 - mae: 0.0500 - val_loss: 0.0022 - val_mae: 0.0365
Epoch 49/100
10/10 ————— 0s 10ms/step - loss: 0.0035 - mae: 0.0469 - val_loss: 0.0037 - val_mae: 0.0497
Epoch 50/100
10/10 ————— 0s 11ms/step - loss: 0.0042 - mae: 0.0487 - val_loss: 0.0022 - val_mae: 0.0363
Epoch 51/100
10/10 ————— 0s 12ms/step - loss: 0.0045 - mae: 0.0544 - val_loss: 0.0040 - val_mae: 0.0538
Epoch 52/100
10/10 ————— 0s 12ms/step - loss: 0.0036 - mae: 0.0472 - val_loss: 0.0016 - val_mae: 0.0312
Epoch 53/100
10/10 ————— 0s 12ms/step - loss: 0.0059 - mae: 0.0576 - val_loss: 0.0022 - val_mae: 0.0395
Epoch 54/100
10/10 ————— 0s 12ms/step - loss: 0.0042 - mae: 0.0526 - val_loss: 0.0025 - val_mae: 0.0424
Epoch 55/100
10/10 ————— 0s 12ms/step - loss: 0.0034 - mae: 0.0421 - val_loss: 0.0027 - val_mae: 0.0439
Epoch 56/100
10/10 ————— 0s 10ms/step - loss: 0.0032 - mae: 0.0471 - val_loss: 0.0018 - val_mae: 0.0342
Epoch 57/100
10/10 ————— 0s 10ms/step - loss: 0.0036 - mae: 0.0465 - val_loss: 0.0018 - val_mae: 0.0334
Epoch 58/100
10/10 ————— 0s 20ms/step - loss: 0.0038 - mae: 0.0450 - val_loss: 0.0014 - val_mae: 0.0270
Epoch 59/100
10/10 ————— 0s 23ms/step - loss: 0.0039 - mae: 0.0502 - val_loss: 0.0023 - val_mae: 0.0397
Epoch 60/100
10/10 ————— 0s 17ms/step - loss: 0.0029 - mae: 0.0423 - val_loss: 0.0020 - val_mae: 0.0358
Epoch 61/100
10/10 ————— 0s 14ms/step - loss: 0.0042 - mae: 0.0507 - val_loss: 0.0019 - val_mae: 0.0363
Epoch 62/100
10/10 ————— 0s 10ms/step - loss: 0.0028 - mae: 0.0410 - val_loss: 0.0018 - val_mae: 0.0343
Epoch 63/100
10/10 ————— 0s 13ms/step - loss: 0.0048 - mae: 0.0539 - val_loss: 0.0040 - val_mae: 0.0560
Epoch 64/100
10/10 ————— 0s 15ms/step - loss: 0.0035 - mae: 0.0460 - val_loss: 0.0018 - val_mae: 0.0307
Epoch 65/100


```

10/10 ----- 0s 13ms/step - loss: 0.0041 - mae: 0.0471 - val_loss:
0.0013 - val_mae: 0.0289
Epoch 66/100
10/10 ----- 0s 13ms/step - loss: 0.0046 - mae: 0.0512 - val_loss:
0.0015 - val_mae: 0.0338
Epoch 67/100
10/10 ----- 0s 12ms/step - loss: 0.0029 - mae: 0.0441 - val_loss:
0.0033 - val_mae: 0.0496
Epoch 68/100
10/10 ----- 0s 12ms/step - loss: 0.0028 - mae: 0.0412 - val_loss:
0.0024 - val_mae: 0.0420
Epoch 69/100
10/10 ----- 0s 14ms/step - loss: 0.0036 - mae: 0.0467 - val_loss:
0.0025 - val_mae: 0.0424
Epoch 70/100
10/10 ----- 0s 11ms/step - loss: 0.0042 - mae: 0.0468 - val_loss:
0.0039 - val_mae: 0.0541
Epoch 71/100
10/10 ----- 0s 14ms/step - loss: 0.0022 - mae: 0.0397 - val_loss:
0.0039 - val_mae: 0.0549
Epoch 72/100
10/10 ----- 0s 13ms/step - loss: 0.0037 - mae: 0.0444 - val_loss:
0.0025 - val_mae: 0.0439
Epoch 73/100
10/10 ----- 0s 10ms/step - loss: 0.0023 - mae: 0.0343 - val_loss:
0.0020 - val_mae: 0.0389
Epoch 74/100
10/10 ----- 0s 16ms/step - loss: 0.0017 - mae: 0.0306 - val_loss:
0.0019 - val_mae: 0.0365
Epoch 75/100
10/10 ----- 0s 13ms/step - loss: 0.0037 - mae: 0.0485 - val_loss:
0.0023 - val_mae: 0.0387
1/1 ----- 0s 467ms/step
[[12.453342]
 [12.900891]
 [14.122293]
 [12.862242]
 [11.147466]
 [13.187765]
 [10.895766]
 [10.757637]
 [12.382955]
 [13.146511]
 [13.985876]
 [11.86746 ]
 [10.137571]
 [13.208725]
 [11.422796]
 [10.8483 ]
 [12.494798]
 [11.275045]
 [10.222591]
 [13.680462]]

MSE: 0.0275
RMSE: 0.1659
R²: 0.9843
MAE: 0.1340

```



```
In [5]: import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import mean_absolute_error

# Load dataset
file_path = "C:/Users/Anarghya/OneDrive/Desktop/major project/energy_consumption_data.csv"
df = pd.read_csv(file_path)

# Encode categorical variable (Weather)
df["Weather"] = LabelEncoder().fit_transform(df["Weather"])

# Select features and target
X = df.drop(columns=["Total Energy Consumption (kWh)"])
y = df["Total Energy Consumption (kWh)"]

# Normalize data
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1))

# Reshape for LSTM (samples, time steps, features)
X_lstm = np.reshape(X_scaled, (X_scaled.shape[0], 1, X_scaled.shape[1]))

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_lstm, y_scaled, test_size=0.2, random_state=42)

# Define LSTM model
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(1, X_train.shape[2])),
    Dropout(0.2),
    LSTM(32, return_sequences=False),
    Dense(1)
])
model.compile(optimizer=Adam(), loss='mse')
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=75, callbacks=[EarlyStopping(monitor='val_loss', patience=10)])
```

```

Dropout(0.2),
Dense(16, activation='relu'),
Dense(1)
])

# Compile model
model.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae'])

# Train model with early stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
history = model.fit(X_train, y_train, epochs=100, batch_size=8, validation_data=(X_test, y_test))

# Predictions
y_pred_scaled = model.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred_scaled)
y_test_inv = scaler_y.inverse_transform(y_test)

# Display Prediction
print(y_pred)

# Evaluation Metrics
mse = mean_squared_error(y_test_inv, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_inv, y_pred)
mae = mean_absolute_error(y_test_inv, y_pred)

print(f"\nMSE: {mse:.4f}\nRMSE: {rmse:.4f}\nR²: {r2:.4f}\nMAE: {mae:.4f}")

mape = np.mean(np.abs((y_test_inv - y_pred) / y_test_inv)) * 100
print("MAPE: ", mape)

residuals = y_test_inv.flatten() - y_pred.flatten()
plt.figure(figsize=(8, 4))
plt.scatter(range(len(residuals)), residuals, alpha=0.6)
plt.axhline(0, color='red', linestyle='--')
plt.title("Residuals of Predictions")
plt.xlabel("Sample")
plt.ylabel("Residual (Actual - Predicted)")
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Training vs Validation Loss')
plt.show()

plt.figure(figsize=(6, 6))
plt.scatter(y_test_inv, y_pred, alpha=0.7)
plt.plot([min(y_test_inv), max(y_test_inv)], [min(y_test_inv), max(y_test_inv)], 'r')
plt.xlabel("Actual Energy Consumption (kWh)")
plt.ylabel("Predicted Energy Consumption (kWh)")
plt.title("Actual vs Predicted Energy (Parity Plot)")
plt.grid(True)
plt.show()

model.summary()

```

Epoch 1/100

```
C:\WINDOWS\system32\myenv\lib\site-packages\keras\src\layers\rnn\rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.  
    super().__init__(**kwargs)
```

10/10 ————— 3s 49ms/step - loss: 0.2510 - mae: 0.4447 - val_loss: 0.2213 - val_mae: 0.3989
Epoch 2/100

10/10 ————— 0s 13ms/step - loss: 0.1188 - mae: 0.2892 - val_loss: 0.0960 - val_mae: 0.2560
Epoch 3/100

10/10 ————— 0s 13ms/step - loss: 0.0544 - mae: 0.1985 - val_loss: 0.0598 - val_mae: 0.2025
Epoch 4/100

10/10 ————— 0s 16ms/step - loss: 0.0667 - mae: 0.2100 - val_loss: 0.0609 - val_mae: 0.2137
Epoch 5/100

10/10 ————— 0s 22ms/step - loss: 0.0401 - mae: 0.1711 - val_loss: 0.0700 - val_mae: 0.2311
Epoch 6/100

10/10 ————— 0s 15ms/step - loss: 0.0638 - mae: 0.2014 - val_loss: 0.0602 - val_mae: 0.2153
Epoch 7/100

10/10 ————— 0s 13ms/step - loss: 0.0352 - mae: 0.1455 - val_loss: 0.0576 - val_mae: 0.2116
Epoch 8/100

10/10 ————— 0s 21ms/step - loss: 0.0357 - mae: 0.1522 - val_loss: 0.0566 - val_mae: 0.2111
Epoch 9/100

10/10 ————— 0s 23ms/step - loss: 0.0426 - mae: 0.1669 - val_loss: 0.0528 - val_mae: 0.2045
Epoch 10/100

10/10 ————— 0s 15ms/step - loss: 0.0468 - mae: 0.1716 - val_loss: 0.0462 - val_mae: 0.1916
Epoch 11/100

10/10 ————— 0s 15ms/step - loss: 0.0211 - mae: 0.1083 - val_loss: 0.0422 - val_mae: 0.1848
Epoch 12/100

10/10 ————— 0s 14ms/step - loss: 0.0198 - mae: 0.1107 - val_loss: 0.0329 - val_mae: 0.1614
Epoch 13/100

10/10 ————— 0s 12ms/step - loss: 0.0168 - mae: 0.1039 - val_loss: 0.0314 - val_mae: 0.1596
Epoch 14/100

10/10 ————— 0s 13ms/step - loss: 0.0172 - mae: 0.1073 - val_loss: 0.0196 - val_mae: 0.1251
Epoch 15/100

10/10 ————— 0s 15ms/step - loss: 0.0137 - mae: 0.0905 - val_loss: 0.0145 - val_mae: 0.1078
Epoch 16/100

10/10 ————— 0s 15ms/step - loss: 0.0089 - mae: 0.0798 - val_loss: 0.0101 - val_mae: 0.0844
Epoch 17/100

10/10 ————— 0s 15ms/step - loss: 0.0101 - mae: 0.0785 - val_loss: 0.0091 - val_mae: 0.0829
Epoch 18/100

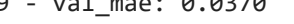
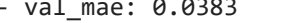
10/10 ————— 0s 13ms/step - loss: 0.0105 - mae: 0.0847 - val_loss: 0.0084 - val_mae: 0.0782
Epoch 19/100

10/10 ————— 0s 18ms/step - loss: 0.0106 - mae: 0.0780 - val_loss: 0.0078 - val_mae: 0.0764
Epoch 20/100

10/10 ————— 0s 16ms/step - loss: 0.0082 - mae: 0.0788 - val_loss: 0.0069 - val_mae: 0.0628
Epoch 21/100

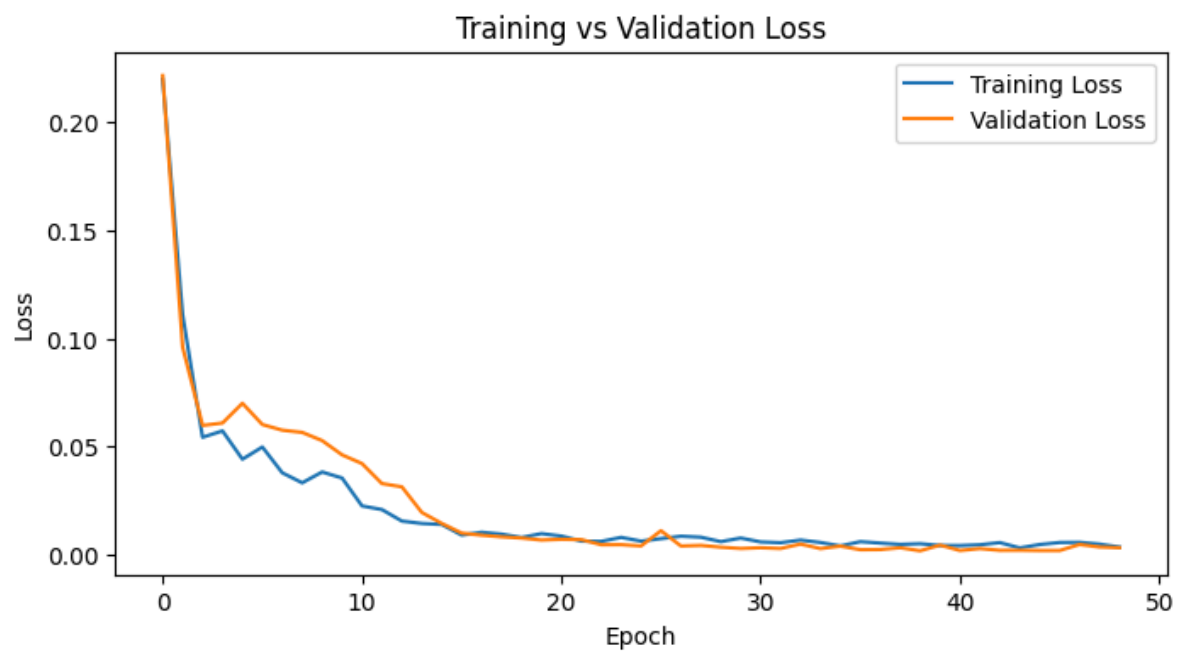
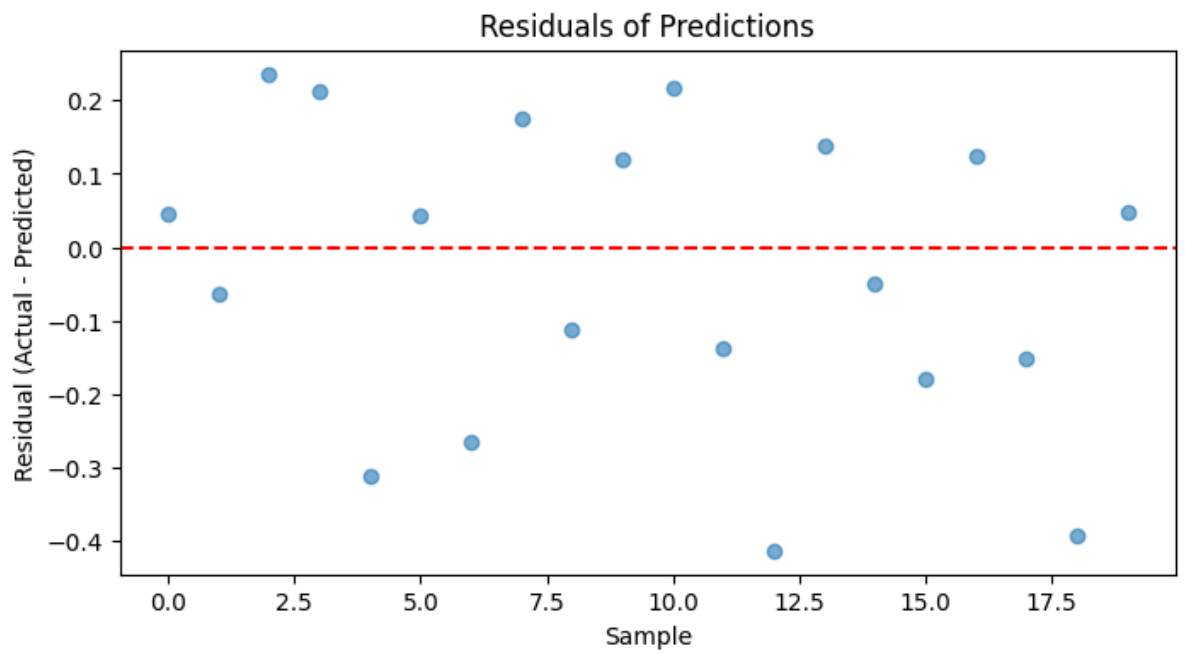
10/10 ————— 0s 17ms/step - loss: 0.0090 - mae: 0.0810 - val_loss: 0.0072 - val_mae: 0.0751
Epoch 22/100

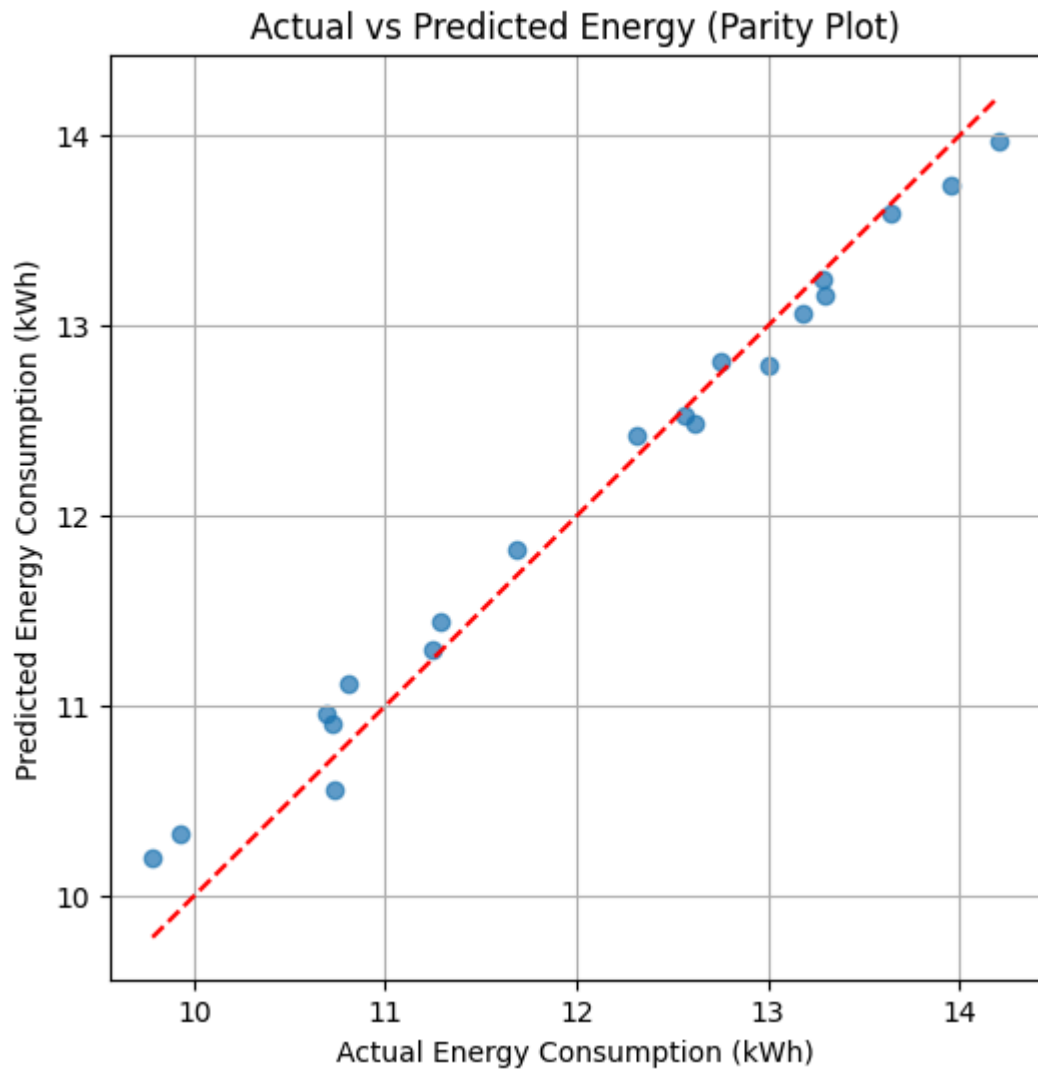
10/10 ————— 0s 13ms/step - loss: 0.0057 - mae: 0.0572 - val_loss:

0.0071 - val_mae: 0.0741
Epoch 23/100
10/10  0s 14ms/step - loss: 0.0068 - mae: 0.0701 - val_loss: 0.0047 - val_mae: 0.0557
Epoch 24/100
10/10  0s 13ms/step - loss: 0.0087 - mae: 0.0777 - val_loss: 0.0047 - val_mae: 0.0576
Epoch 25/100
10/10  0s 11ms/step - loss: 0.0058 - mae: 0.0602 - val_loss: 0.0041 - val_mae: 0.0503
Epoch 26/100
10/10  0s 12ms/step - loss: 0.0064 - mae: 0.0641 - val_loss: 0.0112 - val_mae: 0.0910
Epoch 27/100
10/10  0s 12ms/step - loss: 0.0112 - mae: 0.0822 - val_loss: 0.0041 - val_mae: 0.0547
Epoch 28/100
10/10  0s 11ms/step - loss: 0.0098 - mae: 0.0760 - val_loss: 0.0044 - val_mae: 0.0569
Epoch 29/100
10/10  0s 11ms/step - loss: 0.0065 - mae: 0.0597 - val_loss: 0.0036 - val_mae: 0.0505
Epoch 30/100
10/10  0s 12ms/step - loss: 0.0077 - mae: 0.0681 - val_loss: 0.0030 - val_mae: 0.0459
Epoch 31/100
10/10  0s 15ms/step - loss: 0.0057 - mae: 0.0636 - val_loss: 0.0033 - val_mae: 0.0491
Epoch 32/100
10/10  0s 14ms/step - loss: 0.0051 - mae: 0.0564 - val_loss: 0.0030 - val_mae: 0.0457
Epoch 33/100
10/10  0s 12ms/step - loss: 0.0078 - mae: 0.0700 - val_loss: 0.0050 - val_mae: 0.0617
Epoch 34/100
10/10  0s 12ms/step - loss: 0.0059 - mae: 0.0597 - val_loss: 0.0029 - val_mae: 0.0471
Epoch 35/100
10/10  0s 11ms/step - loss: 0.0044 - mae: 0.0553 - val_loss: 0.0041 - val_mae: 0.0564
Epoch 36/100
10/10  0s 11ms/step - loss: 0.0061 - mae: 0.0608 - val_loss: 0.0024 - val_mae: 0.0414
Epoch 37/100
10/10  0s 9ms/step - loss: 0.0047 - mae: 0.0553 - val_loss: 0.0025 - val_mae: 0.0431
Epoch 38/100
10/10  0s 11ms/step - loss: 0.0046 - mae: 0.0516 - val_loss: 0.0033 - val_mae: 0.0507
Epoch 39/100
10/10  0s 12ms/step - loss: 0.0053 - mae: 0.0579 - val_loss: 0.0019 - val_mae: 0.0370
Epoch 40/100
10/10  0s 10ms/step - loss: 0.0045 - mae: 0.0546 - val_loss: 0.0045 - val_mae: 0.0575
Epoch 41/100
10/10  0s 9ms/step - loss: 0.0041 - mae: 0.0562 - val_loss: 0.0021 - val_mae: 0.0383
Epoch 42/100
10/10  0s 9ms/step - loss: 0.0049 - mae: 0.0548 - val_loss: 0.0029 - val_mae: 0.0478
Epoch 43/100
10/10  0s 13ms/step - loss: 0.0054 - mae: 0.0597 - val_loss: 0.0021 - val_mae: 0.0403

```
Epoch 44/100
10/10 ----- 0s 13ms/step - loss: 0.0029 - mae: 0.0399 - val_loss:
0.0022 - val_mae: 0.0417
Epoch 45/100
10/10 ----- 0s 11ms/step - loss: 0.0054 - mae: 0.0577 - val_loss:
0.0020 - val_mae: 0.0409
Epoch 46/100
10/10 ----- 0s 13ms/step - loss: 0.0051 - mae: 0.0536 - val_loss:
0.0020 - val_mae: 0.0346
Epoch 47/100
10/10 ----- 0s 13ms/step - loss: 0.0065 - mae: 0.0667 - val_loss:
0.0048 - val_mae: 0.0589
Epoch 48/100
10/10 ----- 0s 12ms/step - loss: 0.0062 - mae: 0.0595 - val_loss:
0.0037 - val_mae: 0.0539
Epoch 49/100
10/10 ----- 0s 10ms/step - loss: 0.0033 - mae: 0.0473 - val_loss:
0.0033 - val_mae: 0.0476
1/1 ----- 0s 320ms/step
[[12.52364 ]
 [12.815146 ]
 [13.968702 ]
 [12.787664 ]
 [11.113915 ]
 [13.24408 ]
 [10.961837 ]
 [10.555752 ]
 [12.419481 ]
 [13.062588 ]
 [13.743894 ]
 [11.824956 ]
 [10.197058 ]
 [13.159237 ]
 [11.292415 ]
 [10.906489 ]
 [12.4899435]
 [11.441031 ]
 [10.326621 ]
 [13.594144 ]]

MSE: 0.0410
RMSE: 0.2024
R2: 0.9766
MAE: 0.1718
MAPE: 1.5014962151461813
```





Model: "sequential_3"

Layer (type)	Output Shape	
lstm_6 (LSTM)	(None, 1, 64)	
dropout_6 (Dropout)	(None, 1, 64)	
lstm_7 (LSTM)	(None, 32)	
dropout_7 (Dropout)	(None, 32)	
dense_6 (Dense)	(None, 16)	
dense_7 (Dense)	(None, 1)	

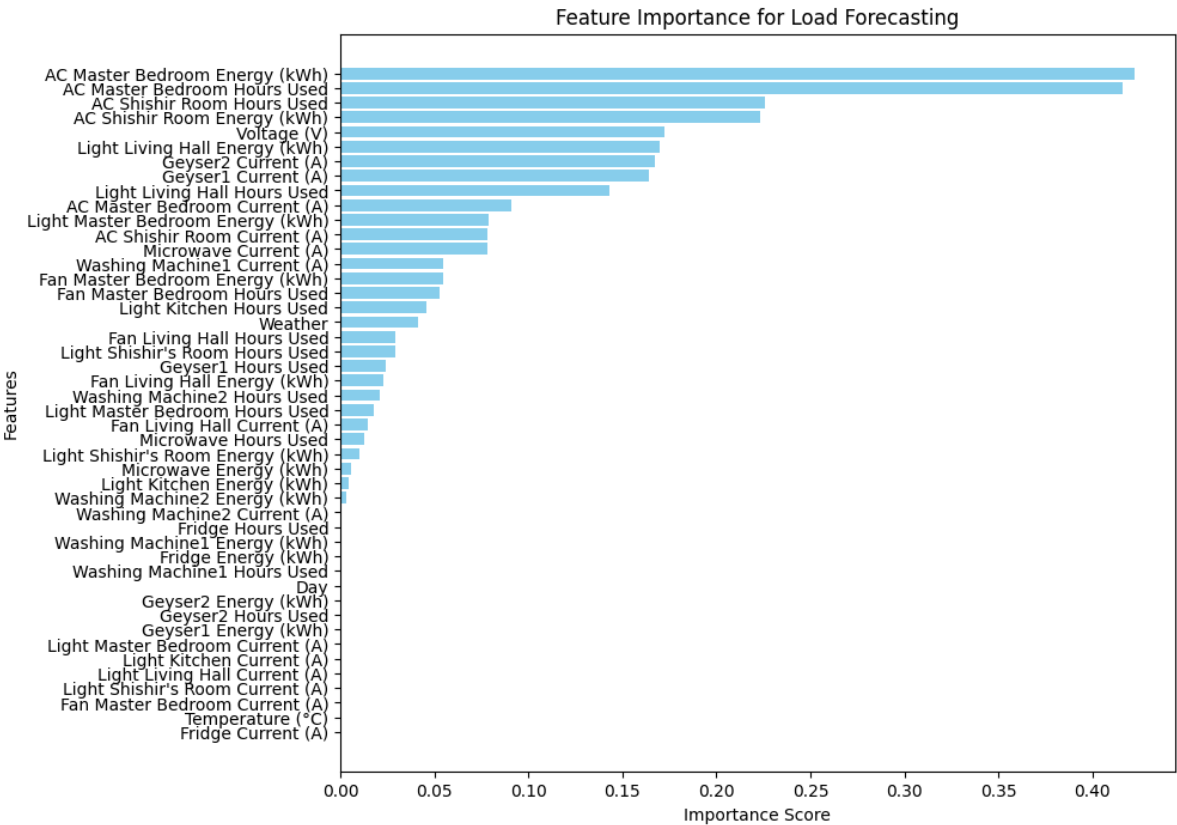
Total params: 124,133 (484.90 KB)

Trainable params: 41,377 (161.63 KB)

Non-trainable params: 0 (0.00 B)

```
In [4]: # Plot feature importance
plt.figure(figsize=(9, 8))
plt.barh(feature_importance_df["Feature"], feature_importance_df["Importance"], col
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.title("Feature Importance for Load Forecasting")
```

```
plt.gca().invert_yaxis()  
plt.show()
```



In []: